

# flashjet

GPU-native jet clustering with the full merge history

Chirayu Gupta

ML4Jets 2026 · Vienna

September 2026

Benchmarks: **ATLAS Open Data** (CC0). Closures: analytic predictions on toy showers.

- 1 Jet clustering, and flashjet
- 2 Correctness
- 3 Speed
- 4 Downstream: the clustering tree as tagger input

## Jet clustering, and flashjet

---

# Reconstruction is moving to accelerators — clustering has not

## Reconstruction is going heterogeneous

- CMS runs GPUs in the HLT since Run 3
- **~30 %** of HLT reconstruction offloaded
- $\rightarrow$  **~25 %** less total HLT time

pixel local reco · pixel tracks + vertices · calorimeter local reco

## Ported so far

- tracking · vertexing · calorimetry
- not jet clustering**

## Why it is still on the CPU

- sequential recombination is **IRC safe** — why it survived
- but each merge **depends on the previous one**
- that data dependence does not map onto a GPU in the obvious way

## Where it hurts

- reclustering **inside a training loop**
- scanning the **jet radius**
- **substructure** on demand

each one pays a host  $\leftrightarrow$  device transfer, over and over

## An open-source GPU jet clusterer

- $k_t$ , Cambridge–Aachen, anti- $k_t$
- custom **Triton** kernels

## Why Triton

- the GPU language of the PyTorch ecosystem
- **compiles and autotunes** for the hardware at hand
- **no separate CUDA build**, no device code to maintain

## Execution

- one program clusters **one event on chip**
- the batch goes out in **one fused launch**
- jets, histories, substructure → **tensors**

## What makes it tractable

Cacciari–Salam NN lemma: the global  $d_{ij}$  minimum sits at some particle's **geometric** nearest neighbour.

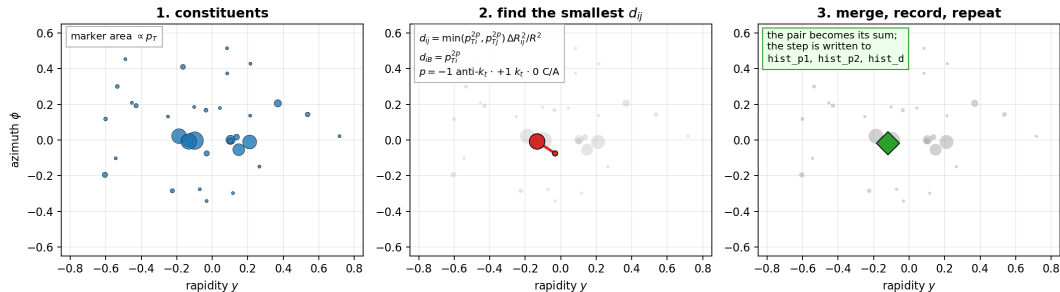
- each slot carries its own NN
- a merge invalidates only  $O(1)$  rows
- $\Rightarrow O(N^2)$ , **not**  $O(N^3)$

## Engineering

- bandwidth-bound → NN update and **next argmin** fused into one sweep
- **branchless** pair-vs-beam (masked stores)
- autotuned **once per GPU model**, cached → **reproducible**

# How sequential recombination works

## Sequential recombination — and what flashjet keeps from it



Repeat until nothing is left: if the smallest distance is a  $d_{ij}$ , merge that pair; if it is a  $d_{iB}$ , call that a jet and remove it. One exponent  $p$  selects the algorithm — the *same* kernel serves all three.

# One exponent, three algorithms

The generalised- $k_t$  distance measure:

$$d_{ij} = \min\left(p_{Ti}^{2p}, p_{Tj}^{2p}\right) \frac{\Delta R_{ij}^2}{R^2}, \quad \Delta R_{ij}^2 = (y_i - y_j)^2 + (\phi_i - \phi_j)^2$$

$$d_{iB} = p_{Ti}^{2p}$$

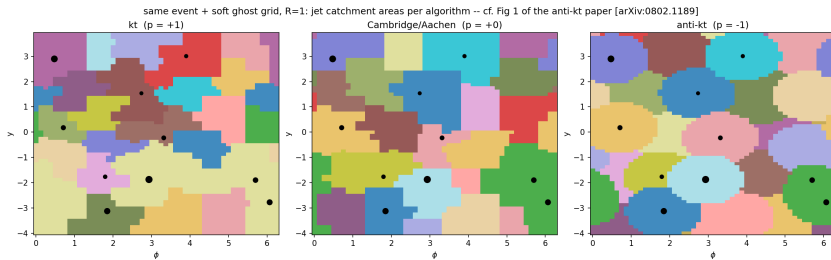
Merge the pair with the smallest  $d_{ij}$ ; if the smallest distance is a  $d_{iB}$ , call  $i$  a jet and remove it.

**Only the exponent  $p$  changes:**

| $p$ | algorithm        | merges first                        | the tree is ordered by           |
|-----|------------------|-------------------------------------|----------------------------------|
| -1  | anti- $k_t$      | around the <b>hardest</b> particles | — (gives the jets)               |
| 0   | Cambridge–Aachen | the <b>closest</b> pair             | <b>angle</b>                     |
| +1  | $k_t$            | the <b>softest</b> pair             | <b>relative <math>p_T</math></b> |

The ordering is **what the substructure reads**: C/A is the tree for **grooming** and the **Lund plane**,  $k_t$  for **exclusive subjects**.

# anti- $k_t$ gives rigid, circular jets



Jet catchment areas for the three algorithms, from flashjet's own clustering.  $k_t$  and C/A give ragged, fluctuating boundaries; **anti- $k_t$  gives circles of area  $\pi R^2$  around each hard particle** — the defining result of the algorithm, recovered here without reference to FastJet.



## It returns the tree, not just the jets

Clustering a batch returns **two** things — the jets, and the record of how they were built.

| field            | shape    | what it holds                           |
|------------------|----------|-----------------------------------------|
| jet_idx          | $(B, N)$ | which jet each particle ended up in     |
| n_jets           | $(B,)$   | how many jets were found                |
| hist_p1, hist_p2 | $(B, N)$ | <b>which pair merged</b> , at each step |
| hist_child       | $(B, N)$ | what they merged <i>into</i>            |
| hist_d           | $(B, N)$ | the distance $d_{ij}$ at that step      |

### Why the second half matters

The bottom three rows are the **complete binary merge tree**. Groomed mass,  $z_g$ ,  $R_g$ , Lund coordinates and exclusive subjets are all **coordinates on that tree** — so they become array reads, with **no second clustering pass**.

# Using it

```
import flashjet

out = flashjet.cluster(p4, mask, R=1.0, algorithm="antikt")
```

| argument  | type               |                                                   |
|-----------|--------------------|---------------------------------------------------|
| p4        | $(B, N, 4)$ tensor | $(p_x, p_y, p_z, E)$ ; CPU or CUDA                |
| mask      | $(B, N)$ bool      | marks real constituents in a padded batch         |
| R         | float              | jet radius                                        |
| algorithm | str                | "antikt"   "kt"   "cambridge"                     |
| p         | float              | generalised- $k_t$ exponent (overrides algorithm) |

|                                              |           |                                         |
|----------------------------------------------|-----------|-----------------------------------------|
| out.n_jets                                   | # (B,)    | jets found                              |
| out.jets_p4(p4)                              | # (B,J,4) | jet four-momenta                        |
| out.splitting_scales()                       | #         | $\sqrt{d_{12}}$ , $\sqrt{d_{23}}$ , ... |
| out.exclusive_jets(n_jets=3)                 | # F1      | exclusive $k_t$ subjects                |
| out.groomed_jets(p4, R, z_cut=0.1, beta=0.0) | # F2      | soft drop / mMDT                        |
| out.lund_coordinates(p4, R)                  | # F3      | primary Lund plane                      |

The same call runs unchanged on CPU and GPU — the backend is chosen from the problem size, not by the caller.

# What is implemented

## Clustering

- generalised- $k_t$ : anti- $k_t$ ,  $k_t$ , C/A
- batched, ragged input via a mask
- three backends, auto-dispatched by size
- merge history recorded in-kernel
- jet regime *and* event regime

## Substructure — reads of the history

- **F1** exclusive  $k_t$  subjects
- **F2** soft drop / mMDT / mass-drop
- **F3** primary Lund coordinates

All three are pure-torch post-reads: no kernel changes, CPU/GPU identical, negligible cost.

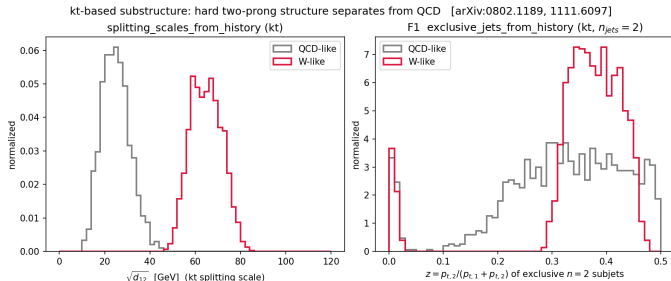
**What it does.** Undo the last merges of the sequence to expose exactly  $n$  subjects — or stop at a  $d_{cut}$ .

**How.** The history is already a binary tree. Walk back up it  $n - 1$  steps; no reclustering.

**Why  $k_t$ .**  $k_t$  merges the softest pair first, so the *last* merges are the hard prongs — exactly what a 2-prong decay leaves behind.

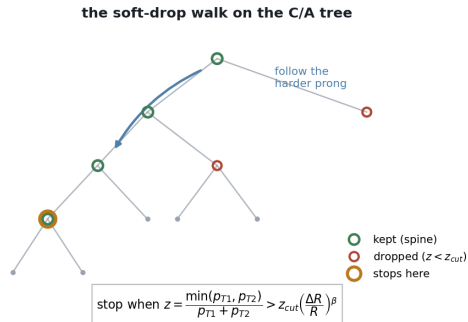
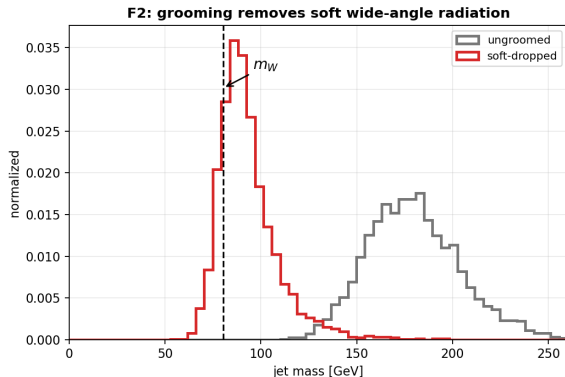
The splitting scales come straight from `hist_d`:

$$\sqrt{d_{12}}, \sqrt{d_{23}}, \dots \text{ with } d_{ij} = \min(p_{Ti}^2, p_{Tj}^2) \Delta R_{ij}^2 / R^2$$



Toy W-like (2-prong) vs QCD-like fat jets. **Left:** the  $k_t$  splitting scale  $\sqrt{d_{12}}$  separates cleanly. **Right:** the momentum balance  $z$  of the two exclusive subjects — the W sits near  $z \sim 0.4$ , QCD spreads to small  $z$ .

## F2 — soft drop / mMDT grooming from the C/A history



Walk down the C/A tree along the harder branch, discarding the softer prong until the momentum-sharing condition is met.  $\beta = 0$  recovers mMDT.

The walk is  $O(\log_2 n)$  on the stored tree — it never touches the constituents again, so grooming is essentially free once clustered.

## C/A merge trees of real jets — root at top, constituents at the bottom

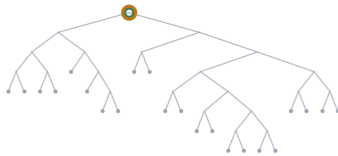
ATLAS Top Tagging Open Data · vertical = declustering depth · horizontal = angular ordering

Light quark / gluon jet



a long spine: soft prong after soft prong  
is stripped, and the mass collapses  
 $n_{\text{drop}} = 8$        $m: 80 \rightarrow 1 \text{ GeV}$

Boosted top — a clean two-prong core



the very first split is already balanced,  
so nothing is groomed away  
 $n_{\text{drop}} = 0$        $m: 80 \rightarrow 80 \text{ GeV}$

Boosted top — the full  $t \rightarrow bW$  system



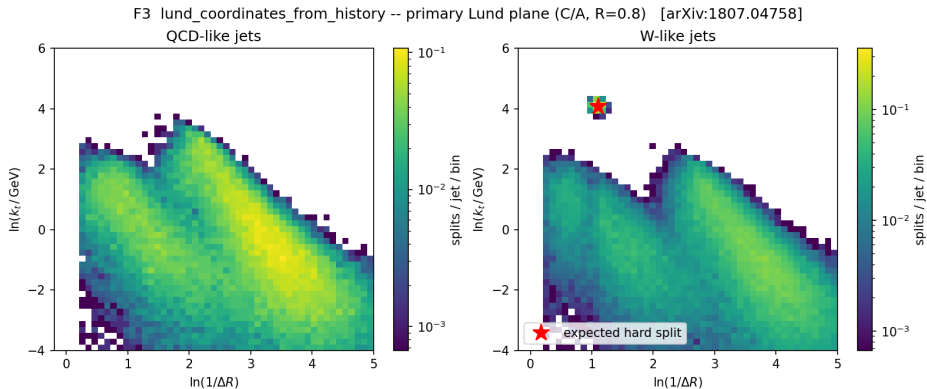
also stops at once, but on a wider split,  
so the whole decay is kept  
 $n_{\text{drop}} = 0$        $m: 170 \rightarrow 170 \text{ GeV}$

● soft-drop spine    ● dropped prong    ● grooming stops here    ● constituent

**A jet without a decay has nothing to stop on.** The light-quark/gluon jet is a long *staircase*: soft prong after soft prong is stripped, and its mass collapses to nothing.

ATLAS Open Data, C/A with soft drop ( $z_{\text{cut}}=0.1$ ,  $\beta=0$ ). **The number of drops is itself a discriminant** — and it is read straight off the stored tree, at no extra cost.

**A real decay stops at once.** Both top jets halt on their very first split, because it is already hard and balanced — so the mass survives.



**What.** Every primary splitting emits  $z = \frac{p_{T2}}{p_{T1} + p_{T2}}$ ,  $\Delta R$ ,  $k_t = p_{T2} \Delta R$ , and the plane coordinates  $(\ln \frac{1}{\Delta R}, \ln k_t)$ .

QCD fills the soft-collinear region smoothly; the W-like sample adds a **localised hard splitting** exactly where the decay is predicted to sit (star).

**How.** One pass down the primary branch — each node is one point in the plane.

**Check.** The  $d$ -channel ties *exactly* to the splitting scales computed independently.

**Correctness**

---



# Two kinds of validation

## First: toys, where truth is known

On generated inputs the *answer* is known analytically — so any observable landing off the predicted curve is a real bug, with nothing to hide behind.

**Input A — toy fat jets** — QCD-like and W-like fat jets

**Input B — toy parton shower** — leading-log parton shower

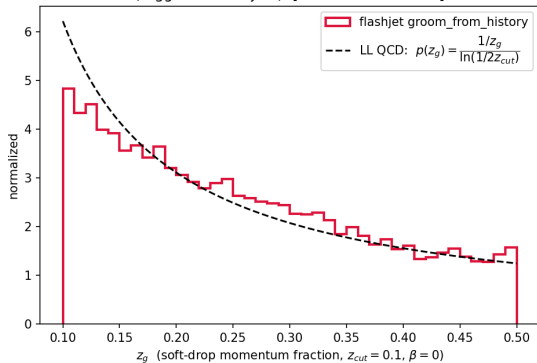
## Then: real data, against FastJet

On detector-level jets there is no analytic answer — so the reference is **FastJet on the same events**, and the experiment's own reconstructed jets.

**Input C — ATLAS Open Data** — large- $R$  jets and full events

The order matters: **toys first** pins the algorithm against a known answer; **real data second** shows it survives detector-level input. A failure at either stage means something different.

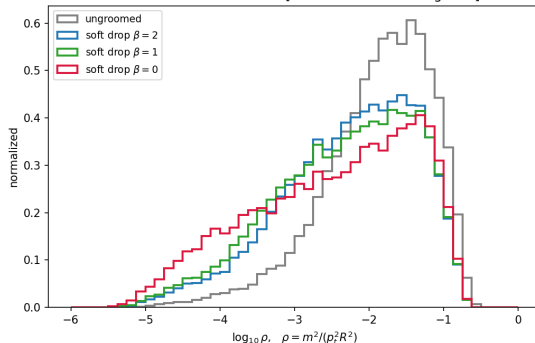
groomed splitting fraction vs the analytic prediction  
(tagged 72% of jets) [cf. arXiv:1402.2657]



Soft-drop  $z_g$  follows the  $1/z$  form predicted at leading-log, recovered from the C/A history.

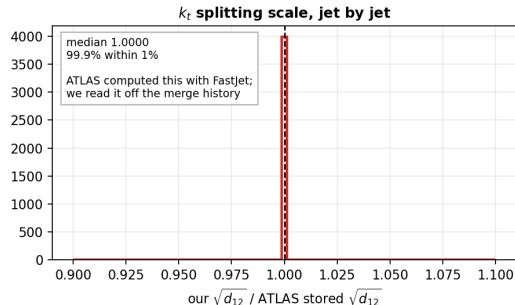
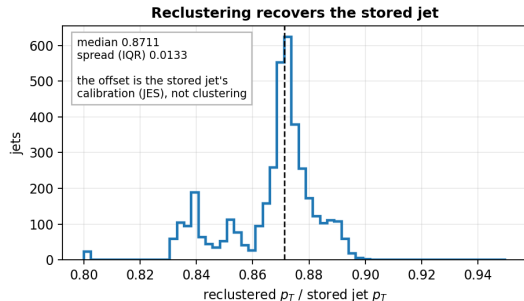
Toy-shower input, so the *prediction* is unambiguous — these test the decoders, not an event generator.

groomed jet mass: stronger grooming (smaller  $\beta$ ) pushes the soft-radiation mass down [cf. arXiv:1402.2657 Figs 3-4]



Groomed-mass ordering in  $\beta$ : larger  $\beta$  grooms less, so the mass peak moves up.

## Jet-by-jet closure against ATLAS-stored quantities



**Left.** Reclustering the stored constituents recovers the stored jet with a **0.013 spread**. The 0.87 offset is the stored jet's **calibration**, applied after clustering — it is present before we touch anything.

**Right.** The  $k_t$  splitting scale  $\sqrt{d_{12}}$  against the value **ATLAS computed with FastJet** and stored: median **1.0000**, **99.9 %** within 1%. We read ours straight off the merge history.

**3 algorithms**  $\times$  **5 radii**  $\times$  **5 multiplicity bins**  $\times$  **2 samples**, 20 000 jets per bin, on ATLAS Open Data.

| check                              | result                                      |
|------------------------------------|---------------------------------------------|
| number of jets found               | <b>100.000 %</b> at every one of 150 points |
| leading-jet $p_T$ within $10^{-4}$ | 1.0000 at 148/150 points                    |
| median relative difference         | $\sim 7 \times 10^{-8}$ (float32 precision) |

### The grid is the point

A single working point proves little. Agreement holds across **all three algorithms**, **every radius** from 0.4 to 1.2, and **24 to 123** constituents per jet.

### Two independent references

The same events are clustered by **FastJet** for the jets, and every substructure decoder is separately pinned against an **independent NumPy tree-walk**.

A stronger test than agreeing with FastJet.

ATLAS publishes an open dataset carrying both the **calorimeter clusters** and **its own reconstructed jets**. Cluster the  $\sim 600$  clusters per event ourselves, and compare:

---

|                            |                      |
|----------------------------|----------------------|
| $n_{\text{jets}}$ vs ATLAS | <b>100.0 %</b> match |
| mean jets/event            | 10.96 vs 10.96       |

---

This closes the algorithm against **the experiment's own published output** — not against another implementation of the same algorithm.

599.3 clusters/event: the event regime, where the problem is  $O(N^2)$ .

**Speed**

---

# The benchmark

**Data** — ATLAS Open Data, freely presentable

| sample             | jets   | $\langle n_{\text{const}} \rangle$ |
|--------------------|--------|------------------------------------|
| boosted top        | 30 000 | 59.2                               |
| QCD (light $q/g$ ) | 30 000 | 52.4                               |

Large- $R$  jets with calorimeter-cluster constituents. The grid is **3 algorithms**  $\times$  **5 radii**  $\times$  **5 multiplicity bins**, 20 000 jets per bin.

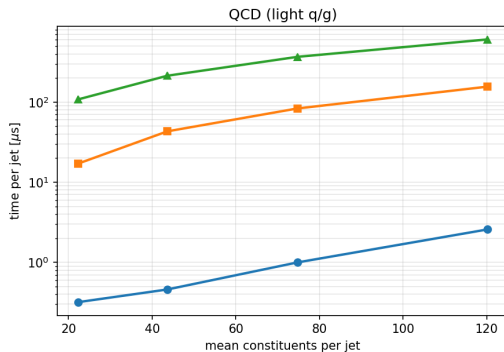
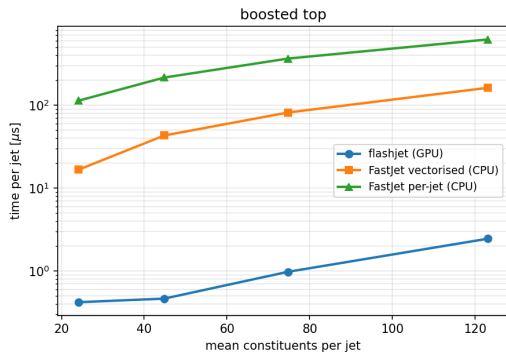
## Method

- 1 extract constituents once, save;
- 2 cluster on GPU, time it, save the *same* events;
- 3 cluster those saved events with FastJet;
- 4 **check they agree** — else the timing is meaningless.

Both clusterers read the **same saved file**. Speedups are quoted against the **vectorised** FastJet interface — the faster, fairer baseline.

# Time per jet

ATLAS Top Tagging Open Data — anti- $k_t$   $R = 1.0$

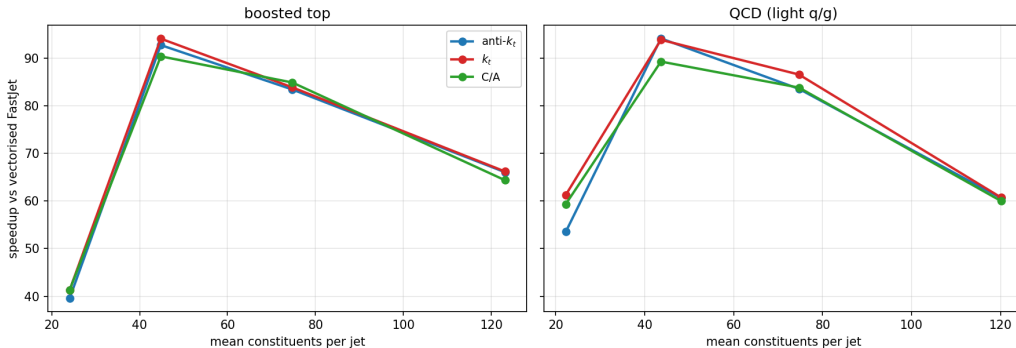


**Log scale** — each gridline is 10×. flashjet stays nearly flat while both FastJet interfaces climb steeply with multiplicity: **0.35–2.7 μs per jet** against **17–155 μs** for vectorised FastJet, and **110–620 μs** per jet one at a time. Benchmarked on an NVIDIA V100-class GPU.



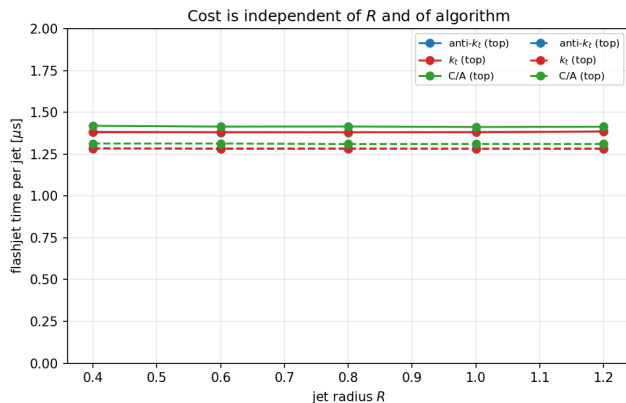
# The advantage grows with jet complexity

Speedup vs multiplicity —  $R = 1.0$  (all points 100%  $n_{jets}$  agreement)



All three algorithms, both samples,  $R = 1.0$ . **39–99 $\times$**  over vectorised FastJet across the grid (median **64 $\times$** ), and every point has **100 %**  $n_{jets}$  agreement. Busier jets  $\rightarrow$  bigger win — the regime that matters for boosted-object physics.

# Cost is independent of $R$ — and of the algorithm



Across  $R = 0.4 \rightarrow 1.2$ : a **0.3 % spread** over a  $3\times$  range in  $R$ .

$R$  changes *which* pairs merge, not *how many* distances are computed.

Across algorithms: anti- $k_t$ ,  $k_t$  and C/A agree to within  $\sim 4\%$  at every multiplicity.

$k_t$  and C/A cost the same as anti- $k_t$ .

**Downstream: the clustering tree as tagger input**

---

[ to be filled in ]

ParT training setup

[ to be filled in ]

ParT training result

- 1 **Clusters on the GPU and keeps the full merge history**, so substructure — exclusive subjects, soft drop, Lund coordinates — comes free as array reads.
- 2 **Gives the same jets as FastJet.** 150/150 grid points at 100 %  $n_{\text{jets}}$  agreement, across anti- $k_t/k_t$ /C-A,  $R = 0.4\text{--}1.2$  and 24–123 constituents per jet.
- 3 **Reproduces the experiment's own reconstructed jets** exactly, at  $\sim 600$  clusters per event.
- 4 **39–99 $\times$  faster** than vectorised FastJet, and the advantage *grows* with jet complexity. Cost is independent of  $R$  and of algorithm.
- 5 Still a **prototype**, with meaningful headroom left.

**Thank you**

Questions?

Benchmarks: ATLAS Open Data (CC0).

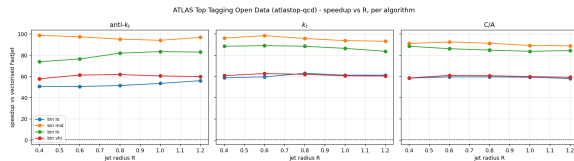
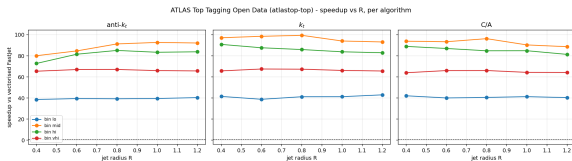
Physics closures: analytic predictions on toy showers.

## Backup

---



# Backup — speedup vs $R$ , per algorithm



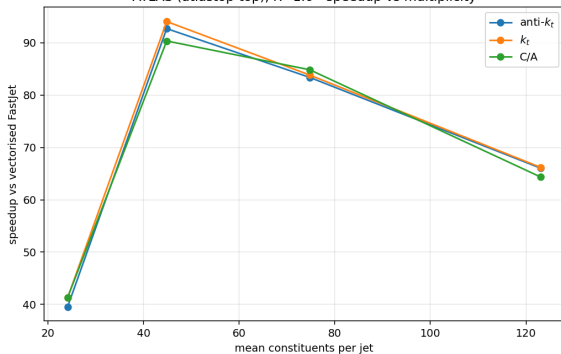
boosted top

Flat in  $R$  for all three algorithms; ordering is by multiplicity bin.

QCD (light  $q/g$ )

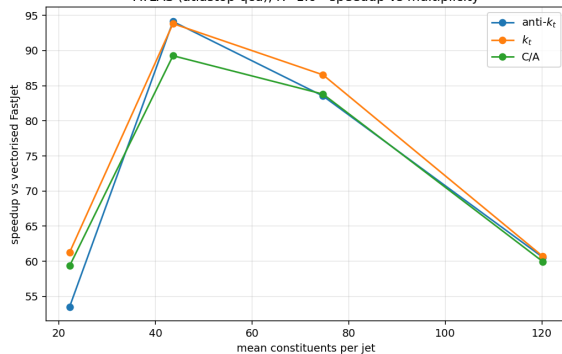
# Backup — speedup vs multiplicity

ATLAS (atlastop-top), R=1.0 - speedup vs multiplicity



boosted top

ATLAS (atlastop-qcd), R=1.0 - speedup vs multiplicity



QCD (light  $q/g$ )

## Backup — the numbers

| sample | alg         | $\langle n \rangle$ | flashjet<br>[ $\mu$ s/jet] | FastJet vec.<br>[ $\mu$ s/jet] | speedup     |
|--------|-------------|---------------------|----------------------------|--------------------------------|-------------|
| top    | anti- $k_t$ | 24.1                | 0.427                      | 16.5                           | 39×         |
| top    | anti- $k_t$ | 44.8                | 0.464                      | 43.0                           | <b>93</b> × |
| top    | C/A         | 74.7                | 1.000                      | 84.8                           | 85×         |
| top    | C/A         | 123.1               | 2.565                      | 165.1                          | 64×         |
| qcd    | anti- $k_t$ | 22.2                | 0.346                      | 17.5                           | 51×         |
| qcd    | anti- $k_t$ | 43.6                | 0.463                      | 45.7                           | <b>99</b> × |
| qcd    | anti- $k_t$ | 74.7                | 1.215                      | 89.8                           | 74×         |
| qcd    | anti- $k_t$ | 120.2               | 2.673                      | 154.9                          | 58×         |

**39–99**× over vectorised FastJet (median 64×), with 100 % jet-count agreement everywhere. Benchmarked on an **NVIDIA V100-class** GPU.

$R = 1.0$  rows shown. Against the per-jet FastJet interface the gap is roughly *two orders of magnitude*.

## Backup — where the time goes

### Nsight Systems — share of GPU time

| kernel                | % GPU       | avg/call   |
|-----------------------|-------------|------------|
| _cluster_large_kernel | <b>97.6</b> | 5.47 ms    |
| _decode_kernel        | 1.5         | 82 $\mu$ s |
| torch glue            | < 1         | —          |

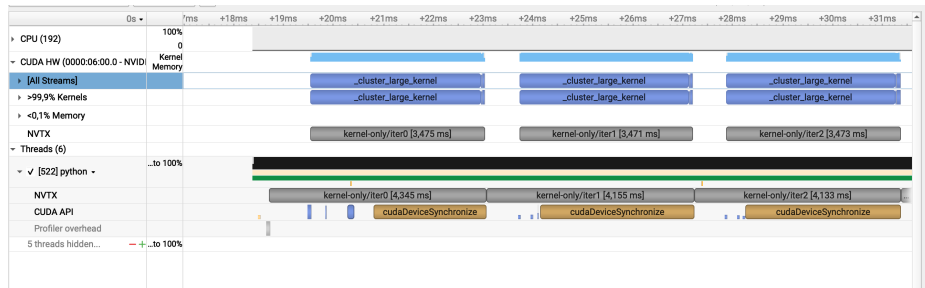
**No hidden overhead** — essentially all the time is real clustering work, and the memory row of the timeline is empty.

### Nsight Compute — hardware counters

| metric                  | value         |
|-------------------------|---------------|
| DRAM throughput         | <b>0.02 %</b> |
| Compute (SM) throughput | 15.4 %        |
| Registers / thread      | 168           |
| Theoretical occupancy   | 18.75 %       |
| Achieved occupancy      | <b>6.25 %</b> |
| Waves per SM            | <b>0.32</b>   |

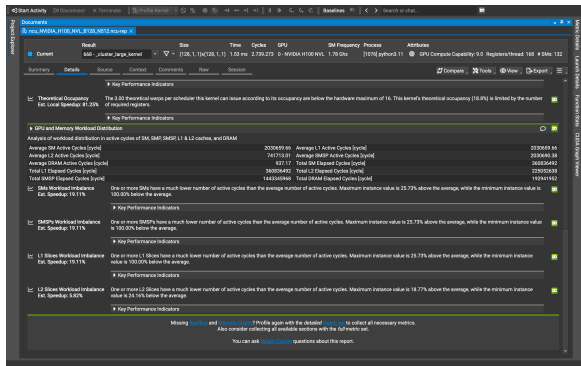
Not memory- or compute-bound — the GPU is simply **not filled**.

# Backup — the GPU timeline



Nsight Systems, three consecutive iterations. Each block is one clustering call; the tool labels the rows “>99.9 % Kernels” and “<0.1 % Memory”. The kernels run back to back with nothing in between and the memory row is empty — no host/device shuffling while clustering runs. That is what makes the speedup real rather than an artefact of where the timer was started.

# Backup — the profiler names the bottleneck



## The tool's own speedup estimates:

finding est. speedup

Theoretical occupancy (registers) 81 %

Achieved occupancy 67 %

SM workload imbalance 19 %

*"This kernel grid is too small to fill the available resources on this device, resulting in only 0.32 full waves across all SMs."*

Grid size **128 blocks** on a **132-SM** device — fewer blocks than the GPU has processors.

~**1.7–1.8×** more available from launch configuration and register budget — **not** from the algorithm.

# Backup — occupancy detail

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |       |                                |    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|--------------------------------|----|
| Occupancy                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |       |                                |    |
| Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicate highly imbalanced workloads. |       |                                |    |
| Theoretical Occupancy (%)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | 18.75 | Block Limit Registers [block]  | 3  |
| Theoretical Active Warps per SM [warp]                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 15    | Block Limit Shared Mem [block] | 6  |
| Achieved Occupancy (%)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 6.25  | Block Limit Warps [block]      | 16 |
| Achieved Active Warps per SM [warp]                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | 4.00  | Block Limit SM [block]         | 32 |
| Cluster Occupancy (%)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 0     | Block Limit Registers [block]  | 32 |
| Max Active Clusters [cluster]                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 0     | Max Cluster Size [block]       | 8  |
| Overall GPU Occupancy (%)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | 0     |                                |    |
| Achieved Occupancy: The difference between calculated theoretical (18.75%) and measured achieved occupancy (6.25%) can be the result of warp scheduling overheads or workload imbalances during the kernel execution. Load imbalances can occur between warps within a block as well as across blocks of the same kernel. See the <a href="#">Occupancy Best Practices</a> page for more details on optimizing occupancy.                                                                                                                                             |       |                                |    |
| Key Performance Indicators                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |       |                                |    |
| Theoretical Occupancy: The 3.00 theoretical warps per scheduler this kernel can issue according to its occupancy are below the hardware maximum of 16. This kernel's theoretical occupancy (18.8%) is limited by the number of required registers.                                                                                                                                                                                                                                                                                                                    |       |                                |    |
| Key Performance Indicators                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |       |                                |    |

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

nvccsmc

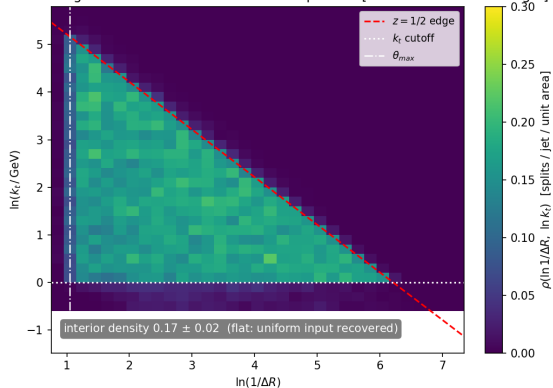
nvccsmc

nvccsmc

nvccsmc

Register pressure (168/thread) caps theoretical occupancy at 18.75 %; achieved is 6.25 %. With 0.32 waves/SM the kernel does not fill the GPU even once, while DRAM throughput is 0.02 % — this is parallelism starvation, not a bandwidth problem. Both are configuration inherited from smaller GPUs, not properties of the algorithm.

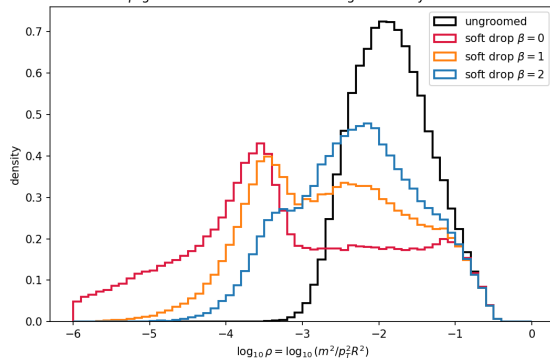
primary Lund plane of the toy shower (C/A, R=0.4):  
m-in-triangle emissions come back as a flat plateau [cf. arXiv:1807.04758 Fig 2]



The Lund plane is populated **uniformly** inside the kinematic **triangle** — the leading-log expectation.

Independent NumPy tree-walks pin every decoder, and the  $d$ -channel of the Lund output ties *exactly* to the splitting scales computed separately.

Soft-drop  $\beta$ -ordering on REAL QCD jets (164292 jets,  $z_{cut} = 0.1$ )  
smaller  $\beta$  grooms harder — same ordering as the toy-shower closure



The soft-drop  $\beta$ -family orders as predicted: larger  $\beta$  grooms less.



## Backup — Input A: the toy fat jets

**Purpose.** QCD vs W is a *known* truth, so any observable that fails to separate them — or lands off the analytic curve — is a real bug.

Two single-fat-jet samples at  $p_T \approx 500$  GeV,  $R = 0.8$ :

- **QCD-like:** one hard collinear core (92 % of  $p_T$ ,  $\sigma_{y,\phi} = 0.06$ ) + soft wide-angle radiation (8 %,  $\sigma = 0.30$ )
- **W-like:** two hard prongs with pair mass  $m = \sqrt{z(1-z)} p_T$ ,  $\Delta R = 80.4$  GeV,  $z \in [0.30, 0.45]$ , random orientation, + 6 % soft contamination

Each “spray” is  $n$  massive pions with exponential  $p_T$  fractions:

```
frac = rng.exponential(1., n); frac /= frac.sum()
pt    = pt_total * frac
y, phi = rng.normal(y0, sigma, n), rng.normal(phi0, sigma, n)
mt     = np.sqrt(M_PI**2 + pt**2)
```

1500 events per sample. flashjet has **no event generator** — it clusters four-vectors; these toys live in the plot scripts, outside the library.

## Backup — Input B: the toy parton shower

A **primary, fixed-coupling, leading-log** shower: emissions sampled **uniformly in the Lund triangle** with density  $\bar{\alpha}$  per unit  $(\ln 1/\theta, \ln k_t)$  area, off a hard spine.

$$\theta = e^{-u}, \quad k_t = e^v, \quad z = \frac{k_t}{p_{T0} \theta}$$

keep  $v \leq \ln(p_{T0}/2) - u$  (i.e.  $z \leq \frac{1}{2}$ ),  $k_t > k_t^{\min}$

### Why this is the right test

The generator's density is *uniform in the triangle by construction*, so the leading-log predictions — the  $1/z$  form of  $z_g$ , the  $\beta$ -ordering of the groomed mass — are unambiguous. Landing on them tests the **decoders**, not the shower.

**Jet areas** use the anti- $k_t$ -paper construction: one hard event plus a dense grid of **infinitely soft ghosts** ( $p_T = 10^{-8}$ ); the ghosts each algorithm sweeps up trace its catchment area.

$\bar{\alpha} = 0.25$ ,  $p_{T0} = 1$  TeV,  $R = 0.4$ , 20 000 showers; fixed seed, fully re-runnable.

## Backup — the validation ladder

| rung                    | input  | reference                                | result                                  |
|-------------------------|--------|------------------------------------------|-----------------------------------------|
| unit tests              | —      | independent NumPy tree-walks             | 129 pass, incl. GPU paths               |
| anti- $k_t$ jet areas   | toy B  | the anti- $k_t$ paper's own construction | rigid cones, $\rightarrow \pi R^2$      |
| $k_t$ substructure      | toy A  | known 2-prong vs QCD truth               | clean separation                        |
| soft-drop $z_g$         | toy B  | leading-log $1/z$ prediction             | on the curve                            |
| groomed mass vs $\beta$ | toy B  | Soft Drop Figs. 3–4                      | ordering reproduced                     |
| Lund plane              | toy A  | primary-Lund construction                | hard split where predicted              |
| jet-by-jet kinematics   | real C | FastJet, same events                     | 150/150 points, 100 % $n_{\text{jets}}$ |
| full-event clustering   | real C | the experiment's own jets                | 100 % match, $\sim 600$ clusters/ev     |

**Each rung is closed against something independent** — an analytic prediction, a separate NumPy implementation, or FastJet itself. No rung is validated against another part of flashjet.

All benchmark data is **ATLAS Open Data** (CC0), read over the public redirector — no grid certificate required.

| dataset                       | used for                             |
|-------------------------------|--------------------------------------|
| ATLAS Top Tagging Open Data   | timing + FastJet agreement           |
| 2020 ATLAS Jet Reconstruction | closure vs the experiment's own jets |

Constituents are calorimeter clusters, massless. Physics closures use toy showers compared against leading-log analytic predictions, so the prediction being tested is unambiguous.